Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра систем штучного інтелекту



Звіт

до лабораторної роботи № 2

з дисципліни

Проектування інформаційних систем

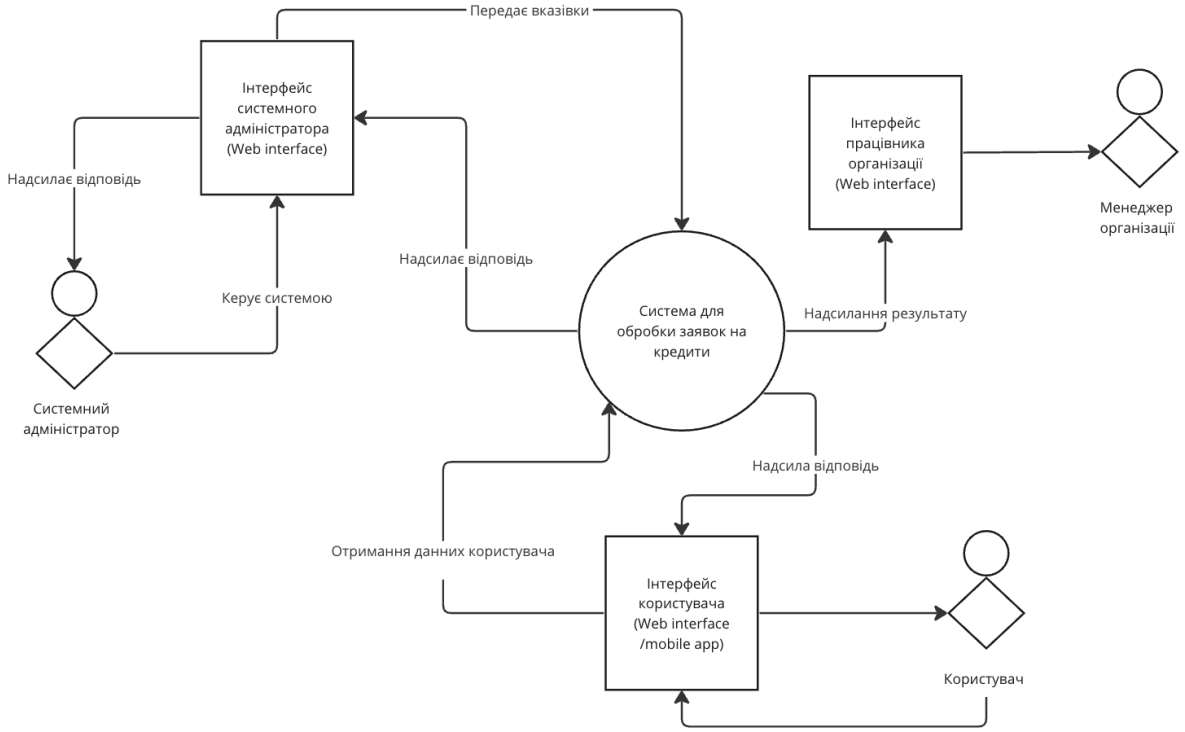
Виконав:
студент КН-315
Лозовий О.А.

Львів – 2023

Architecturally Significant Requirements:

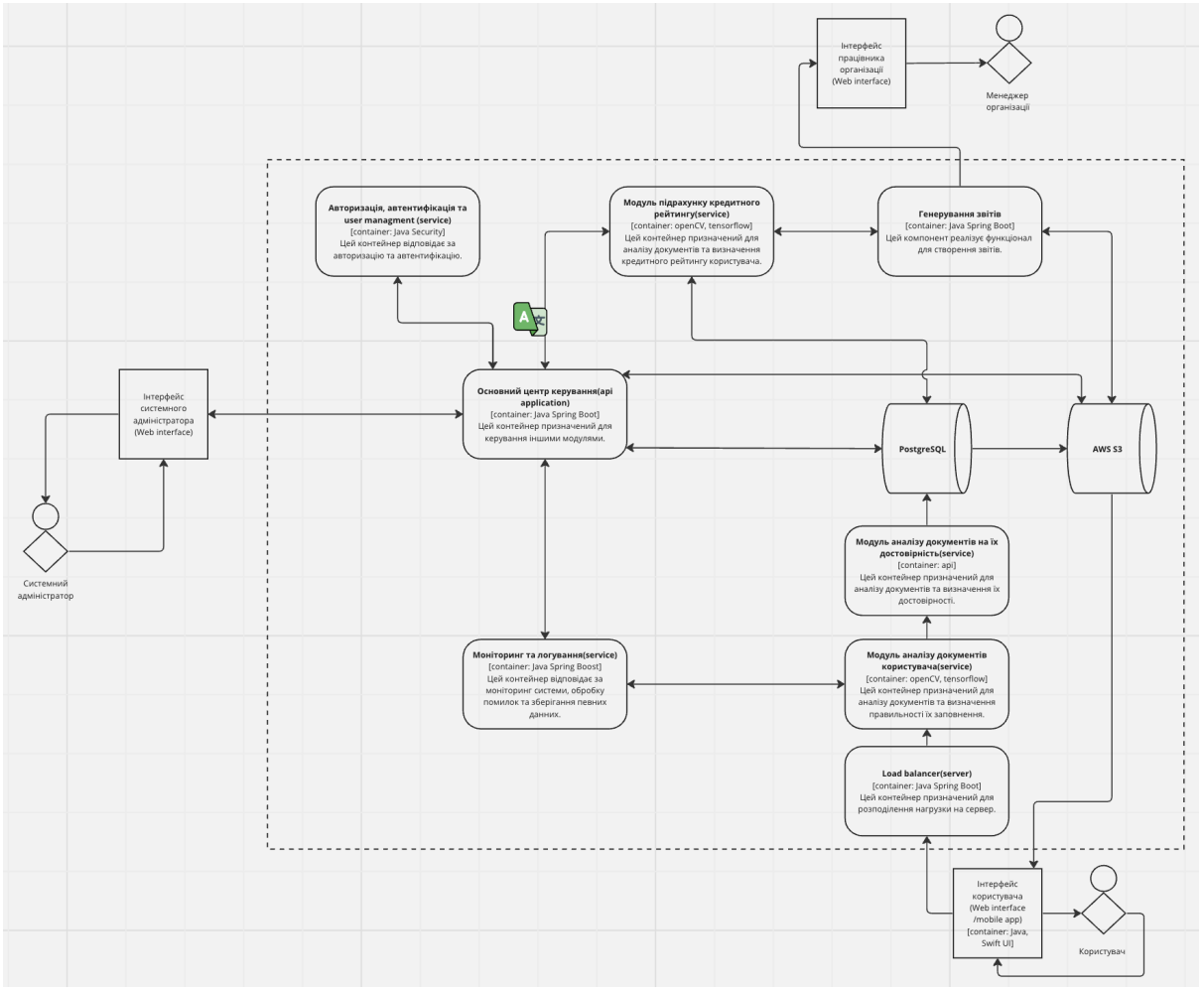
1. Збереження даних
    a. Збереження звітів вимагає таку базу даних, яка легко може збільшуватись разом зі збільшенням кількості звітів.
    b. Дані користувачів також необхідно зберігати (у зашифрованому вигляді).
    c. Для зберігання резервних копій моделі(на випадок збою), необхідно виділити місце на сервері.
2. Система повинна підтримувати можливість збільшення кількості користувачів
    a. Збільшення користувачів приведе до потреби більшої бази даних, яка може вмістити більшу кількість користувачів.
    b. Також при збільшенні кількості запитів, можливі збої в роботі, тому необхідно обмежити трафік при великих навантаженнях.
3. Інтеграції
    a. Для інтеграції з органами, що видають документи, необхідно створити API, або ж інтегрувати зовнішнє API.
    b. Для інтеграції машинного навчання, необхідно створити модель, натренувати її та перевірити точність її роботи.
4. Мультиплатформність
    a. Необхідно створити мобільний додаток для IOS і Android.
    b. Необхідно створити веб-інтерфейс для інших платформ.

Контекстна діаграма:



На даній діаграмі зображено взаємодію системи обробки заявок на кредити з зовнішнім середовищем. В даному випадку система обробки заявок на кредити взаємодіє з Системним адміністратором, Користувачем та Менеджером фінансової організації, за допомогою відповідних інтерфейсів.

Діаграма контейнерів:



Для проектування свого сервісу я обрав мікросервісну архітектуру, адже вона, на мою думку, буде найбільш доречна в даному випадку.

Мікросервіси дозволяють розділити функціональність системи, що дозволяє розробникам працювати над різними частинами сервісу паралельно, що значно зменшить час реалізації проекту.

Через можливість незалежного масштабування окремих компонентів (ASR 2a), ця архітектура дозволяє оптимізувати використання ресурсів, а також забезпечити швидкі відповіді на зміну навантаження.

Також мікросервіси дозволяють враховувати велику кількість користувачів/факторів при розробці алгоритмів обробки даних, прийняття рішень, а за необхідності й зменшувати навантаження на систему для кращої її роботи(ASR 2b).

Варто зазначити, що, якщо конкретний сервіс вийде з ладу, це не призведе до збою всієї системи, що є дуже важливим при роботі з фінансами та документами.

Мій сервіс складається з 8 мікросервісів:

1. Основний центр керування – це api додаток, який дозволяє системному адміністратору, керувати системою, налаштовувати та коригувати її. Також через цей модуль, адміністратор буде мати доступ до інших частин системи, для зміни їх конфігурації. Оскільки база даних теж частина системи, то сисадмін буде мати доступ до її даних(дані зашифровані). Також через цей модуль будуть отримуватись та записуватись дані про помилки.

   Для розробки даного контейнеру я обрав Java Spring Boot. Він підтримує багато різних стандартів та протоколів,має хорошу систему безпеки дозволяє ефективно інтегрувати його з різними системами.
2. Моніторинг та логування – сервіс, призначений для моніторингу та зберігання помилок системи.

   Також розроблений за допомогою Java Spring Boot.
3. Авторизація, автентифікація та user management – сервіс, який призначений для авторизації та автентифікації усіх користувачів (Системний адміністратор, Менеджер фінансової організації, Клієнт фінансової організації), а також визначення базової інформації про користувача, для збереження його даних у базі даних.

   Розроблений за допомогою Java Security, через зручність використання, та надійність.
4. Модуль аналізу документів користувача – сервіс для перевірки правильності документів клієнта.

   Розроблений за допомогою OpenCV та Tensorflow.
5. Модуль аналізу документів на їх достовірність – сервіс для перевірки достовірності документів клієнта.

   Оскільки для фінансової організації важливо, щоб усі документи були справжніми, необхідно перевірити їх правдивість. Це буде зроблено за допомогою інтеграції стороннього API(ASR 3a). Найкраще у даному випадку буде інтегрувати API Дія для запровадження максимальної безпеки даних клієнтів організації, та постійного доступу до API.
6. Модуль підрахунку кредитного рейтингу – сервіс який дозволить створити рейтинг клієнта фінансової організації на

основі отриманих та оброблених нами документів.

Для розробки цього контейнера буде використано Python для роботи з бібліотеками Tensorflow та OpenCV. Через їх швидкодію та можливість створити та натренувати певну модель-алгоритм(ASR 3b), ми отримаємо гарантію точності результату, при необхідній нам швидкодії.

7. Load balancer – сервіс який дозволяє розподілити навантаження між іншими сервісами.

   Також розроблений за допомогою Java Spring Boot.

8. Генерування звітів – сервіс який дозволяє створити звіт про клієнта фінансової організації.

   Для реалізації компоненти обрано Java Spring Boot через його швидкодію та багатий функціонал.

Також варто сказати і про бази даних(ASR 1a, 1b, 1c):

Для усіх потребностей буде використовуватись база даних **PostgreSQL**. Вона вважається однією з найбільш надійних та стабільних реляційних баз даних, підтримує транзакційні операції, що дозволяє уникнути втрати даних у випадку непередбачуваних ситуацій.

В той ж час, для збереження даних у хмарі буде використано **AWS S3** від **Amazon**. Воно гарантує високий рівень доступності та надійності, забезпечує високу швидкість завантаження об'єктів та надає багато інших функцій, набір інструментів, SDK для роботи з S3, що полегшує реалізацію та управління реплікацією даних.

Також варто використовувати **Kubernetes** та **Docker**:

Docker надає можливість упаковувати додатки та їх залежності у контейнери, що забезпечує ізоляцію середовища та допомагає уникнути конфліктів.

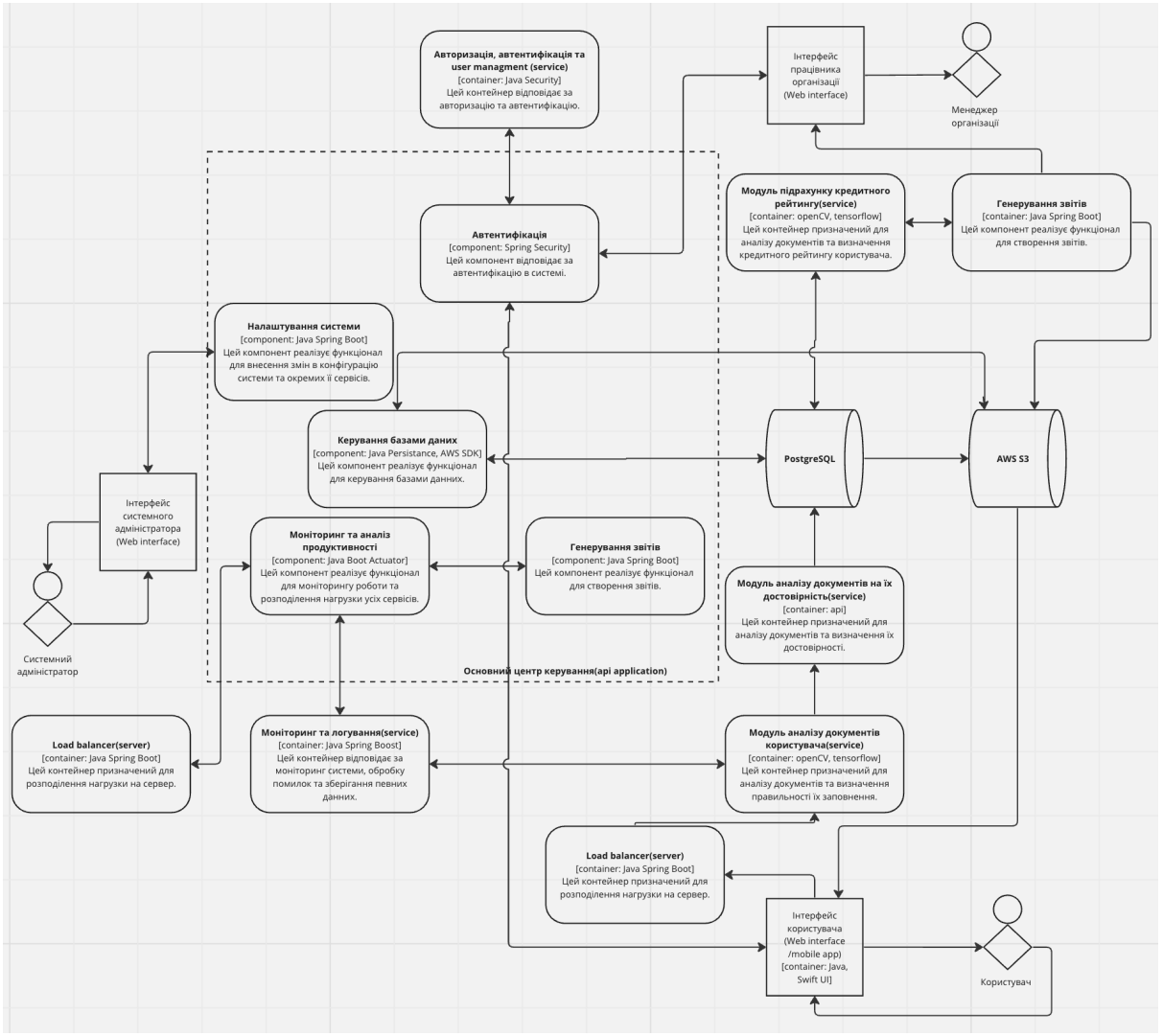
Kubernetes надає засоби для автоматизованого розгортання та масштабування додатків з контейнерів, що сприяє легкості управління великими кластерами.

Docker дозволяє створювати образи додатків для резервування та відновлення, що є важливим для відновлення у випадку аварій.

Docker та Kubernetes надають механізми для управління версіями додатків та їх залежностей, що допомагає впроваджувати оновлення.м

Контейнеризація та Kubernetes дозволяють легко інтегрувати моніторинг та логування, щоб вчасно виявляти проблеми та покращувати продуктивність системи керування трафіком, а також дозволяють швидко розгортати нові версії додатків та впроваджувати їх у виробниче середовище без великих зусиль.

Діаграма компонентів:



На цій діаграмі я детально показав компоненти основного центру керування.

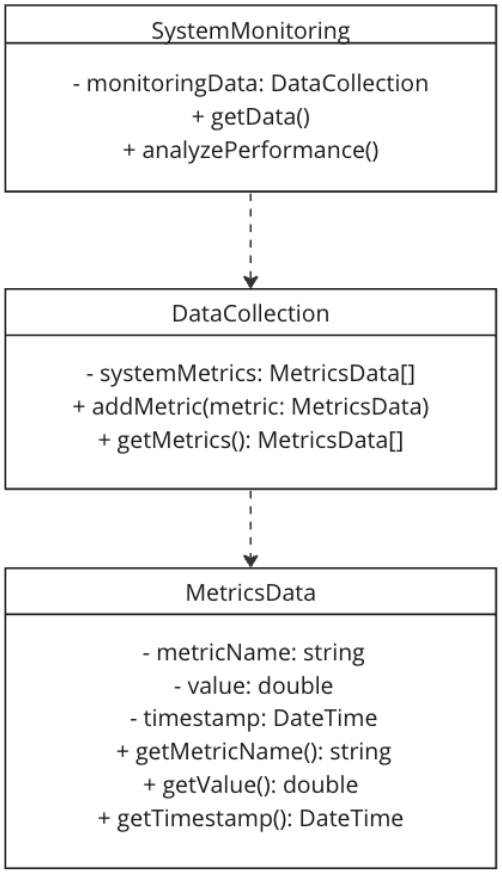
1. Автентифікація – компонент, який відповідає за авторизацію у системі користувачів, де ті можуть робити відповідні їмі дії: Клієнт - подати документи, Менеджер - отримати звіт.

   Реалізація цього компонента буде виконана за допомогою Java Security, оскільки ця технологія гарантує безпеку.

2. Керування базами даних – компонент, який відповідає за керування як основною базою даних так і хмарною.

   Для цього обрано Java Persistence та AWS SDK.
3. Моніторинг та аналіз продуктивності – для реалізації компоненти обрано Java Boot Actuator, оскільки він проводить збір метрик, інформації про стан системи.
4. Генерування звітів – для реалізації компоненти обрано Java Spring Boot через його швидкодію та багатий функціонал.
5. Налаштування системи – для реалізації компоненти обрано Java Spring Boot через його швидкодію, багатий функціонал та можливість масштабування.

Діаграма класів:



Діаграма класів - це графічне представлення структури класів та їх взаємозв'язків у програмній системі. Вона відображає, як класи спілкуються один з одним, що вони роблять та як вони взаємодіють для досягнення певної функціональності.

Ця діаграма відображає структуру компоненту моніторингу та аналізу продуктивності системи та взаємодію між його основними класами.

У цій діаграмі маємо три класи: "SystemMonitoring," "DataCollection" і "MetricsData." Клас "SystemMonitoring" відповідає за моніторинг та аналіз продуктивності системи. Він має поле "monitoringData" типу "DataCollection," яке використовується для зберігання даних моніторингу. "SystemMonitoring" має два методи: "getData," який збирає дані про продуктивність системи, і "analyzePerformance," який аналізує ці дані.

Клас "DataCollection" відповідає за збір і зберігання метрик продуктивності системи. Він має поле "systemMetrics" типу "MetricsData[]" для зберігання різних метрик. Клас "DataCollection" має два методи: "addMetric," який додає нову метрику до колекції, і "getMetrics," який повертає всі зібрані метрики.

Клас "MetricsData" представляє окрему метрику продуктивності. Він має поля для назви метрики ("metricName"), її значення ("value") та мітки часу ("timestamp"). Клас "MetricsData" має методи для отримання цих полів.

Висновок: Під час виконання цієї лабораторної роботи, я розробив технічний дизайн для системи керування трафіком. Я детально вивчив такі діаграми як контекстну, діаграму контейнерів, діаграму компонентів і діаграму класів. Завдяки цій роботі я поглибив свої технічні знання і дізнався, як працюють різні технології.